Start here	2
Working with your AI	3
Stop 1: Warm-up done	7
Stop 2: GitHub connected	9
Stop 3: Hello world pushed	10
Stop 4: PRD shared	14
Stop 5: App live	20
Other ways to take part	25
After the day	33
Every prompt	37
If something goes wrong	44
Stay safe	47
Every link	50

The *take-away*

Everything from the hackathon in one place: how to work with your AI, every step from warm-up to a live app, and every useful prompt and link. It's yours to keep.

Please read this first

This is a personal learning exercise. Use fictional or public sample content only. Do not enter, upload or share any LBG data in AI tools, GitHub repositories, prompts, screenshots or published websites. More on staying safe.

The latest copy is at hackday.bonhard.ai/talks/takeaway.pdf. Organised by Phil Bonhard. This guide does not claim endorsement by any organisation or approval of any tool.

Start here

How to use this

This is everything from the hackathon, written to be read on its own. You don't need the talks, and you can dip in anywhere.

Follow the five stops

The steps run in the same order as the My day board, from warm-up to a live app.

Copy the prompts

On the web page, each prompt has a Copy button. In the PDF, select the text. Change the words in [square brackets] to your own.

Read what comes back

A prompt is a starting point. Adjust the wording to your idea, then read and challenge what the AI gives you.

1. **Warm-up done**
   Hello world on your own computer
2. **GitHub connected**
   Your AI tool can save to your account
3. **Hello world pushed**
   Your page is on GitHub with a live address
4. **PRD shared**
   A short plan, reviewed and saved
5. **App live**
   Someone else can open it on their own device

Talk to it like a colleague

Think of your AI as a very fast colleague who's never met you. It knows a lot about building things and nothing about who you're designing for. That's your job.

You don't need special words. Brief it the way you'd brief a colleague. If the answer isn't right, say so. It won't be offended.

Ask it for help with anything, including "how do I...?"

- **Tell it who you are.** "I'm a designer. Explain things in plain words."
- **Give it context.** Who it's for, the problem, and what good looks like.
- **Keep talking.** Ask follow-up questions, and say "that's not what I meant".
- **Let it question you.** Ask it to ask you questions before it builds anything.

Show it, don’t describe it

A screenshot beats a paragraph. If something looks wrong, or you see an error you don’t understand, grab it, paste it and ask “what’s going on here?”

On a Mac

Grab part of the screen. Press `⌘ Cmd` `⇧ Shift` `4`, then drag a box round the bit you need.

Put it in the chat. Drag the thumbnail (bottom right) into the chat. Or add `Ctrl` to copy it, then press `⌘ Cmd` `V`.

On Windows

Grab part of the screen. Press `⊞ Win` `⇧ Shift` `S`, then drag a box round the bit you need.

Put it in the chat. It’s copied already. Click in the chat and press `Ctrl` `V`.

> **Grab only the bit you need.** A full-screen shot can catch Teams, email, notifications and other tabs. Share only a screenshot of your own sample product.

Stay in charge

AI can be confidently wrong. The cure is simple: small steps, and look at the result every time.

One change at a time

Ask, check, then the next thing.

Check it yourself

Open it, click it, try it on your phone. “Done” doesn’t always mean done.

Save when it works

Publish a working version before a big change.

If in doubt, ask it to explain its choices. “What were the options? Why this one?” That’s how you learn, and it’s often how you catch the mistake.

Going round in circles? Start a fresh chat and explain where you’ve got to.

Permissions, money and getting stuck

These tools can create files, run commands and use your accounts. That’s what makes them useful, and it’s why you read before you say yes.

Read before you say yes

Your AI asks permission before it does things. Say yes to your project folder and GitHub. Say no to anything you don’t understand, then ask for help.

Don’t buy anything on the spot

Asked to pay, upgrade or add a card without planning to? Stop and ask. Ran out of usage? Don’t top up in a panic.

The 10-minute rule

Stuck for ten minutes? Ask for help. Screenshot, paste, ask: your AI is the first helper you go to.

> **Never paste a password, key or token into a chat.** If your AI asks for one, say no. GitHub gives you a one-time code to type in yourself.

Choose your tool

Both do the same job. The warm-up takes about 25–35 minutes the first time (an estimate) and checks that your tools work. It's a rehearsal, not your product.

Claude

In the Claude desktop app's Code tab. Needs Claude Pro or higher, because Claude Code isn't on the free plan. Buying a plan is your choice.

Download the Claude app

Codex

In the ChatGPT desktop app. Listed on free and paid ChatGPT plans, with usage limits, so a paid plan gives you more room.

Download the ChatGPT app

> **Plans change.** Check the provider's own site before you sign up. Use your own personal accounts, not a work account or work email.

Make your first page

1. **Install the app and sign in.** In Claude, click the Code tab. In the ChatGPT app, choose Codex from the dropdown.
2. **Create a free GitHub account** at github.com/signup, with your own email. Your username is public. Remember how you signed up (email, Google or Apple) and always sign in the same way.
3. **Give it an empty folder.** Make a new folder called `hello-hackday` on your Desktop and open it in your tool.
4. **Ask for your page.** Paste this in. It may ask a question or two, or just make the page: either is fine. If it asks, answer in your own words, or say "you decide".

   > I'd like you to create a webpage that simply says hello world.

5. **See it.** Ask "Please open the page in my browser", or double-click the file ending in `.html`. The address starts with `file://`, which means it's only on your computer.

Connect your tool to GitHub

This lets your tool put your work on GitHub with one request. You won't type a password, and your tool won't open a browser. GitHub gives you a short one-time code, and you type it in yourself.

1. **Be signed in to GitHub somewhere,** in your usual browser, or on your phone if that's where you signed up.
2. **Paste this into the same chat.**

> Please connect this computer to my GitHub account, ready for the day. Go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Run gh auth login --hostname github.com --git-protocol https --web in the background, then show me the one-time code clearly. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token.
> 3. When I say done, run gh auth setup-git and gh auth status, and tell me my GitHub username.
> Don't create a repository or publish anything today.

3. **Type the code.** It looks like `ABCD-1234`. Go to github.com/login/device on any device where you're signed in, type it, press Continue, then Authorize. It works for about 15 minutes. Then tell your tool "done".
4. **You're finished when it tells you your GitHub username.** It's fine to approve git and gh, and installing them if it asks. Using Codex? Say yes when it asks to use the internet.

The loop you'll use all day

Everything from here is the same four moves. You'll go round them twice with your hello-world page, then all day with your real project.

1. **Change**
   Ask your AI
2. **Check**
   Look at it yourself
3. **Publish**
   Push to GitHub
4. **Share**
   Open it on a phone

Four words you'll hear

Repository (repo)

Your project's folder, stored on GitHub.

Commit

A save point you can go back to.

Push

Send your saved changes up to GitHub.

GitHub Pages

Turns your repository into a website with a real address.

Put your page online

Your page and its file become public on GitHub. Anyone with the address can see them. That's fine for a sample sentence and not for anything else.

Open your `hello-hackday` folder in your tool. Prefer your own words? Say you want to put this page on GitHub and make it a website, one step at a time. Or paste this.

> I'd like to publish this page on the web with GitHub Pages. It only contains fictional sample content. Please go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Check I'm signed in to GitHub with gh auth status. If I'm not, run gh auth login --hostname github.com --git-protocol https --web in the background and show me the one-time code. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token, and never store one.
> 3. Create a new PUBLIC repository called hello-hackday in my own GitHub account from this folder, commit the page and push it.
> 4. Turn on GitHub Pages for that repository, publishing from the main branch and the root folder.
> 5. Tell me my web address (it looks like https://USERNAME.github.io/hello-hackday/), check whether it is live yet, and tell me clearly if I need to wait.

Check it’s really there

Once your AI says it’s pushed, don’t take its word for it. Open GitHub and look.

1. **Read each request.** It’s fine to approve git and gh (GitHub’s own tool) for your `hello-hackday` folder and repository. If it asks for anything else, say no and ask for help.
2. **See your file on GitHub,** at `github.com/your-username/hello-hackday`. If you can see it, it’s on GitHub.
3. **Open your address on your phone.** It looks like `https://your-username.github.io/hello-hackday/`. Opening it on your phone proves someone else can open it too.

> **A 404 page usually means “not yet”.** GitHub can take up to 10 minutes the first time. Still missing after 15? Screenshot it and ask your AI to check that GitHub Pages is on for `hello-hackday`.

Change it and publish again

This is the moment it clicks: you changed something, checked it, and it's live for anyone.

1. **Ask for one change.** Reload the page on your own computer to check it.

   > Please add a smiley to my page and make it look a bit friendlier.

2. **Ask it to publish the change.**
3. **Refresh the live address.** Give it a minute. Still seeing the old version? Do a hard refresh: `Ctrl` `Shift` `R` on Windows, `⌘ Cmd` `Shift` `R` on a Mac.

> **That's the whole day in miniature:** connected, created, published, shared, changed, published again. Keep `hello-hackday` as your practice repository. Your real project gets a fresh start.

Pick something small

Choose a theme or bring your own idea. Every theme can be a simple website, small enough to finish in a day, and AI features inside your product are optional.

Personal website

A one-page site for a made-up person: who they are and how to get in touch.

Elf Help

A seven-day Elf on the Shelf planner for busy parents and carers.

Mortgage journey

A guided journey for a first-time buyer at a clearly made-up bank.

Your own idea

One user, one job and no more than three essential things it must do.

> **The briefs are on the site.** See the project themes, including two advanced routes for anyone who wants a challenge. Expect to work more on your own on those.

Start fresh

Your warm-up was a rehearsal. Leave it as it is, so your practice page stays safe and your real project stays tidy.

New folder

Somewhere easy to find, named after your project.

New repository

Its own home on GitHub, separate from `hello-hackday`.

New chat

A clean start for your AI, with your idea at the centre.

In Claude or Codex, open the new folder and paste this. Builders such as Lovable connect to GitHub in their own way, so follow their steps.

> I'm starting a new product project in this empty folder. It will only hold fictional or public sample content. Please guide me one step at a time, in plain language, to create a new public GitHub repository for it in my own account and connect this folder to it. Ask before installing anything, and never ask me for a password or token.

> **Working in a pair?** Both of you need to be able to open and edit the project. On GitHub, the owner can invite a partner from the repository's Settings → Collaborators. Swap the keyboard every 25–30 minutes.

Question your idea

Before you build, think the idea through with AI. Allow about 25 minutes for this and the next two steps. The goal is a small, clear plan that you understand and have agreed.

Replace [idea] with a sentence about your idea and paste this in. It works with any assistant, and you don't need to install anything. This part takes about 10 minutes.

> Help me think through this product before we build it. My idea is [idea]. Start with the intended user, their situation and the problem. Ask a few focused questions at a time; wait for my answers before asking questions that depend on them. Challenge vague claims and suggest alternatives, but let me make the decisions. Cover the main user journey, what makes the idea useful, assumptions we cannot yet prove, accessibility, the smallest useful release, what to leave out, and how we will test success. Keep the conversation within about ten minutes; record unresolved questions as assumptions instead of inventing answers. Then propose a concise PRD and ask me to review and agree it before building. Do not generate the implementation yet.

> **Answer in your own words.** If it challenges you, that's the point. Push back when you disagree, and say "I don't know yet" rather than inventing an answer. Unknowns become assumptions to test.

Choose the journey and scope

About 5 minutes. A small plan you can finish beats a big one you can't.

One user

One real-sounding person and the problem they have.

One complete task

Something they can do from start to finish.

Three features at most

Only the essential ones. Decide what you're leaving out, and say why.

> **Set the limits if it hasn't asked.** Tell it: "I need to publish today using [tool/host], with £0 additional service costs unless I explicitly choose otherwise. Use fictional data where needed."

Draft, challenge and save your PRD

A PRD is a short product plan. Let the AI draft it, then read it properly and fix anything that isn't what you meant. Allow about 10 minutes, and aim for one or two pages.

It should cover

- The user, their problem and the outcome you want
- Assumptions you still need to test
- The main user journey
- What's in the first release, and why
- What's left out
- Three to five checks that show it works
- How you'll publish it, and what comes next

Before you build, can you explain?

- **Who needs it, and why.**
- **One complete task,** step by step.
- **Why this first release:** what's in, what's out and the reason.
- **How you'll check it:** the checks from your PRD.

> **Save it as** `PRD.md` in your new project's GitHub repository, not the warm-up one. A list of features with no clear user need is a sign to go back a step.

Share your PRD

On the day, you send your PRD, your app and your product page to the organiser before the demos. It takes two minutes, and it puts your product on the big screen.

1. **Open** Share your PRD and add your name and your project's name. Working in a pair? Put both names.
2. **Add your addresses:** your app, and your product page if you've made one. Not live yet? Leave them blank and send it again later.
3. **Upload your PRD or link to it.** Is it still in a chat? Ask your AI: "Give me the PRD as a Markdown file I can download."

> **Your PRD is never published.** Sharing is open only around the event. After that, keep `PRD.md` in your repository: it's the best way to pick up where you left off.

Build in small slices

Get one piece working, check it against your PRD, then move on. Aim to get a first version online early, then improve it.

> Read the agreed PRD. Propose the smallest first slice we can build and publish now. Explain the files or services you need and any cost before adding dependencies. Implement in small steps, help me preview it and give me concrete checks after each step. Check each slice against the PRD and tell me if anything we learn should change it. Keep a working version before major changes. If a feature needs a backend, do not imply static hosting will run it. Use my chosen publishing route and help me publish early.

Update the plan

When building or testing teaches you something, change the PRD. That's good design, not failure.

Polish isn't proof

A good-looking result doesn't show people need it. Testing tells you what to improve next.

Stop adding features

Leave time to fix what testing finds and to publish again.

Test it with someone else

Someone outside your pair tries the main task. It’s the quickest way to find out what you can’t see for yourself.

1. **Don’t coach.** Give them the address and ask them to attempt the main task.
2. **Watch where they hesitate,** and write down one thing you found.
3. **Tell your AI what happened.** Fill in [observation] and paste this.

> Here is what happened when someone tested my published product: [observation]. Compare it with the PRD acceptance checks. Suggest the smallest useful improvement, explain the trade-off and help me implement and verify it. Do not expand the scope unnecessarily.

4. **Publish again** and check the address on another device.

Make a product page

One simple web page about your product, on its own address, with a button to the real thing. It's the link you send when someone asks what you built. Allow about 15 minutes, after you've fixed the app.

1. **Ask your AI.** Give it your PRD (attach the file, or point it at `PRD.md`), and put your app's address in place of [address].

   > Make a simple, single web page about the product in my PRD. It is separate from the app itself. Include the product's name, who it's for, the problem it solves, what it does in three short points, and a clear button that opens the live app at [address]. Use only what's in the PRD: don't invent features, numbers, people or quotes. Keep it to one page that works well on a phone. Then help me publish it on its own address, one step at a time.

2. **Publish it on its own address,** the same way you published your hello-world page.
3. **Check it on another device,** and check the button opens your app.

The finish line

Six things to have by the end of the day. Check them before the demos.

- ✓ A live web address that someone else can open on their own device.
- ✓ Someone outside your pair has tested it, and you've written down one thing you found.
- ✓ You know how to change it and publish it again.
- ✓ A reviewed `PRD.md` saved in your project's GitHub repository.
- ✓ Everyone who built it can open the source or the editable project.
- ✓ You've chosen one next improvement.

> **What doesn't count.** A screenshot, a preview on your own computer or a link to a builder's editor isn't a published product: other people need to be able to open the live address. If it still isn't live by the end, say so honestly and write down what's left to fix. That's a useful result too.

Show what you made

About three minutes, in front of everyone. No slides needed: show the real thing.

1. **Who it's for** and the problem it solves.
2. **The live product,** opened at its address.
3. **What you did yourself.**
4. **One thing you learned.**
5. **The next improvement** you'd make.

> **Before you show it,** open the published address outside your builder and on another device, and run your PRD's checks. Note anything still incomplete. Working in a pair? Share the time, and each explain your own contribution.

On a phone or tablet

You can do the whole day on a phone or tablet. Typing is harder, but it's a fun way to find out how little you need a computer. Use your own device and personal accounts, not a work phone.

1. **Install the Claude app** from the App Store or Google Play. It works on the free plan, and a paid plan gives you more room.
2. **Ask for the page** in a new chat, then tap the preview and try the button.

   > Please make a single-page website as an artifact, with a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when tapped. Use sample text only. Make it look good on a phone.

3. **Change it with your voice.** Tap the microphone on your keyboard and say what you'd like.
4. **Publish it.** With the artifact open, look for Publish or Share and copy the link. Can't find it? Open claude.ai in your phone's browser and publish from there.
5. **Check someone else can open it.**

Plan, build and publish in one app

You follow the same planning as everyone else and skip the GitHub steps, which are for laptops.

1. **Question your idea.** Start a new chat and paste in the planning prompt. On a paid plan, create a project first so your PRD and chats stay together.
2. **Keep your PRD.** Ask: “Put the agreed PRD in a Markdown document.” Save a copy as `PRD.md` in your Files app.
3. **Build in small slices.** Ask Claude to build the first slice from the PRD as a single-page artifact, check it, then ask for the next.
4. **Publish, and publish again.** After changes, publish again and open the link to check it shows the latest version.

Talk instead of type

Use the microphone for long answers.

Let the AI ask

Say “ask me one question at a time” so your answers can be short.

Power up

Bring a charger and a power bank. Building all day drains a battery.

On an LBG laptop

If your LBG laptop has a Figma Make licence, you can take part on it. You plan with Microsoft Copilot, and build and publish in Figma Make.

Please read this first

You're on an LBG laptop, so LBG's normal rules for this machine and its tools apply. Use fictional or public sample content only, even though you're at work: no customer, colleague or internal information in prompts, files or anything you publish. More on staying safe.

It shares inside LBG only

Anything you publish opens only for colleagues signed in to LBG's Figma.

Your work stays with LBG

Your project lives in LBG's Figma account, so you can't take it home.

No GitHub, no installs

Everything happens in Figma and Copilot.

AI credits

Figma Make uses credits from your licence. How many you have for a full day is To be confirmed

Warm up in Figma Make

About 15 minutes (an estimate). These steps follow Figma's documentation and haven't yet been tried by us on an LBG laptop, so button names may differ slightly.

1. **Open Figma Make.** Open Figma, signed in with your work account as usual, and create a new Make file. Can't see Make? Your licence may not include it, so message Phil on Teams.
2. **Ask for the page,** then click the button in the preview to check it works.

   > Create a single-page website with a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when clicked. Use sample text only. No sign-in, database or other services.

3. **Publish it inside LBG.** Click Publish. If Figma asks who can see it, choose your organisation (LBG only), then copy the address.
4. **Check someone else can open it.** Send the address to a colleague on Teams. They may need to be signed in to Figma.

Plan in Copilot, build in Make

You follow the same planning as everyone else. Only the tools change.

1. **Question your idea with Copilot.** Paste in the planning prompt, answer in your own words and push back when you disagree.
2. **Save your PRD.** Ask Copilot for it, then paste it into Notepad. Choose Save as, set the type to All files, and save it as `PRD.md` in your OneDrive. Word is fine too.

   > Please write the agreed PRD in Markdown, inside a single code block, so I can copy it into a file.

3. **Hand it to Figma Make.** Start a new Make file and paste in your PRD with this on top.

   > Here is the PRD for what we're building. Read it, then tell me the first small slice you'd build and wait for my go-ahead before you start.

4. **Build in small slices.** When you learn something, update `PRD.md` and tell Make what changed.

In the browser with Lovable

Build in your browser and publish with one button. These steps follow Lovable's documentation (checked 7 October 2026) and haven't yet been tried by us. Check the limits before you sign up.

1. **Sign up at lovable.dev** with your personal email. Its documentation lists a Free plan with 5 build credits a day, up to 30 a month. If you're asked for a card, stop.
2. **Ask for the page,** then try the button in the preview.

   > Create a single-page website with a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when clicked. Use sample text only. No sign-in, database or other services.

3. **Publish.** Click Publish at the top right, accept the suggested address and click Publish again. On Free and Pro plans, anyone with the link can open it.
4. **Change it and publish again.** Changes don't go live until you click Publish and Publish changes.
5. **Save it to GitHub.** Open Project settings → Git → GitHub and let Lovable create the repository. It's private by default. Don't delete it, or the link stops working.

By hand: make the page

No installs and no payment, and you do every step yourself. About 30–40 minutes the first time (an estimate). No AI to hand? The do-it-yourself route on the site has a ready-made page to use instead.

1. **Ask any AI assistant for the page.** Copy just the part from `<!DOCTYPE html>` to `</html>`.

   > Write one complete HTML file for a tiny web page. It needs a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when clicked. Put everything in that single file, with no external libraries. Use sample text only, with no personal or work information. Reply with the code only.

2. **Save it as `index.html` on your Desktop.** On a Mac, use TextEdit: choose Format → Make Plain Text before you paste, and pick Use .html if asked. On Windows, use Notepad: set “Save as type” to All files.

3. **Open it in your browser.** Double-click the file. See code instead of a page? It was saved with the wrong type, so check step 2.

By hand: publish on GitHub

Your page and its file will be public on GitHub. That's fine for a sample sentence and not for anything else.

1. **Create a repository** at github.com/new. Name it `hello-hackday`, choose Public and press Create repository.
2. **Upload your page.** Click "uploading an existing file", drag in `index.html` and press Commit changes.
3. **Turn on publishing.** Open Settings, then Pages. Set Source to Deploy from a branch, the branch to `main` and the folder to `/ (root)`, then press Save.
4. **Wait, then open your address.** It looks like `https://your-username.github.io/hello-hackday/` and can take up to 10 minutes.
5. **Change a sentence.** Click `index.html`, then the pencil icon. Edit it and press Commit changes. Wait a minute or two, then reload your address.

After the day

Keep what you built

Your project belongs to you. Before the day fades, write down how to get back to it, so you can open it again in a month's time.

Paste this into the chat where you built it. Then save the README to GitHub.

> Based on the actual project, draft a short README with its purpose, live URL, source/editable-project location, exact steps to reopen, run, edit and publish, known limitations and next three improvements. Include required services and any ongoing costs. Do not include credentials. Distinguish verified instructions from anything still untested. Help me verify the instructions and save the result to GitHub.

- ✓ **Keep your accounts.** Use your own personal accounts, and make sure you keep access to your work.
- ✓ **Keep your repository.** A published address can stop working if you delete what's behind it.

After the day

Pick it up again later

Coming back after a break? Your AI won’t remember the day, but your PRD and README will catch it up.

1. **Open your project’s folder** in your tool, or the project in your builder, and start a new chat.
2. **Catch it up.** Paste this in.

   > Read PRD.md and the README in this project, then tell me in plain words where we got to, what's still incomplete and the next improvement I noted. Suggest the smallest next step, and wait for my go-ahead before changing anything.

3. **Make one change,** check it yourself, then publish again.
4. **Update the plan.** When you learn something, change the PRD.

> **Keep testing with people.** Ask someone to try the main task without coaching, then use the feedback prompt on what you find.

Trying something more ambitious

Databases and outside services are welcome, as long as they stay free of real data. Expect more setup and troubleshooting.

One new thing at a time

Choose a database or an API, not both.

Keep keys secret

Store an API key in your host's secret settings. Never paste it into a prompt, a repository file or code that runs in the browser.

Check costs and limits first

A free API doesn't mean free hosting or instant access. Any spending is your choice.

Have a fallback

If setup stalls, sample data keeps your product working. Label it clearly as sample data.

What travels with you

The tools you tried may not be available on your work machine. What travels with you is how you work: you can carry these habits into your design practice.

- **Frame the problem.** Start with one user and what they need.
- **Direct the AI.** You make the decisions, and you stay responsible for them.
- **Improve it.** Change the plan when you learn something.
- **Question assumptions.** Let the AI challenge you, and write down what you don't know yet.
- **Test the idea.** Watch someone else use it.

> **Before you go:** tell us how the day went, and look out for the event summary.

Prompts: warm-up and GitHub

Ask for your first page

> I'd like you to create a webpage that simply says hello world.

Connect your tool to GitHub

> Please connect this computer to my GitHub account, ready for the day. Go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Run gh auth login --hostname github.com --git-protocol https --web in the background, then show me the one-time code clearly. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token.
> 3. When I say done, run gh auth setup-git and gh auth status, and tell me my GitHub username.
> Don't create a repository or publish anything today.

Prompts: publishing

Put your page online

> I'd like to publish this page on the web with GitHub Pages. It only contains fictional sample content. Please go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Check I'm signed in to GitHub with gh auth status. If I'm not, run gh auth login --hostname github.com --git-protocol https --web in the background and show me the one-time code. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token, and never store one.
> 3. Create a new PUBLIC repository called hello-hackday in my own GitHub account from this folder, commit the page and push it.
> 4. Turn on GitHub Pages for that repository, publishing from the main branch and the root folder.
> 5. Tell me my web address (it looks like https://USERNAME.github.io/hello-hackday/), check whether it is live yet, and tell me clearly if I need to wait.

Change how it looks

> Please add a smiley to my page and make it look a bit friendlier.

Prompts: planning

Start a new project on GitHub

> I'm starting a new product project in this empty folder. It will only hold fictional or public sample content. Please guide me one step at a time, in plain language, to create a new public GitHub repository for it in my own account and connect this folder to it. Ask before installing anything, and never ask me for a password or token.

Question the idea and draft the PRD

> Help me think through this product before we build it. My idea is [idea]. Start with the intended user, their situation and the problem. Ask a few focused questions at a time; wait for my answers before asking questions that depend on them. Challenge vague claims and suggest alternatives, but let me make the decisions. Cover the main user journey, what makes the idea useful, assumptions we cannot yet prove, accessibility, the smallest useful release, what to leave out, and how we will test success. Keep the conversation within about ten minutes; record unresolved questions as assumptions instead of inventing answers. Then propose a concise PRD and ask me to review and agree it before building. Do not generate the implementation yet.

Prompts: building and testing

Build the smallest publishable version

> Read the agreed PRD. Propose the smallest first slice we can build and publish now. Explain the files or services you need and any cost before adding dependencies. Implement in small steps, help me preview it and give me concrete checks after each step. Check each slice against the PRD and tell me if anything we learn should change it. Keep a working version before major changes. If a feature needs a backend, do not imply static hosting will run it. Use my chosen publishing route and help me publish early.

Use real feedback

> Here is what happened when someone tested my published product: [observation]. Compare it with the PRD acceptance checks. Suggest the smallest useful improvement, explain the trade-off and help me implement and verify it. Do not expand the scope unnecessarily.

Prompts: finishing and carrying on

Make a product page

> Make a simple, single web page about the product in my PRD. It is separate from the app itself. Include the product's name, who it's for, the problem it solves, what it does in three short points, and a clear button that opens the live app at [address]. Use only what's in the PRD: don't invent features, numbers, people or quotes. Keep it to one page that works well on a phone. Then help me publish it on its own address, one step at a time.

Write a README

> Based on the actual project, draft a short README with its purpose, live URL, source/editable-project location, exact steps to reopen, run, edit and publish, known limitations and next three improvements. Include required services and any ongoing costs. Do not include credentials. Distinguish verified instructions from anything still untested. Help me verify the instructions and save the result to GitHub.

Pick it up again later

> Read PRD.md and the README in this project, then tell me in plain words where we got to, what's still incomplete and the next improvement I noted. Suggest the smallest next step, and wait for my go-ahead before changing anything.

Prompts: other routes

A page in Lovable or Figma Make

> Create a single-page website with a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when clicked. Use sample text only. No sign-in, database or other services.

A page on a phone

> Please make a single-page website as an artifact, with a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when tapped. Use sample text only. Make it look good on a phone.

A page by hand

> Write one complete HTML file for a tiny web page. It needs a heading that says "Hello, world", one friendly sentence, and a button that changes the sentence when clicked. Put everything in that single file, with no external libraries. Use sample text only, with no personal or work information. Reply with the code only.

Get your PRD out of Copilot

> Please write the agreed PRD in Markdown, inside a single code block, so I can copy it into a file.

Hand your PRD to Figma Make

> Here is the PRD for what we're building. Read it, then tell me the first small slice you'd build and wait for my go-ahead before you start.

Short phrases worth keeping

You don’t need special words, but these do a lot of work.

- ✓ “I’m a designer. Explain things in plain words.”
- ✓ “What were the options? Why this one?”
- ✓ “I don’t know yet.”
- ✓ “What’s going on here?” (with a screenshot)
- ✓ “Give me the PRD as a Markdown file I can download.”
- ✓ “Ask me one question at a time.”
- ✓ “That’s not what I meant.”
- ✓ “You decide.”
- ✓ “Please open the page in my browser.”
- ✓ “Don’t ask for a password or token. Use the one-time code instead.”

When you’re stuck

Everyone gets stuck. It’s part of working this way, and you don’t need to sit with it on your own.

1. **Screenshot, paste, ask.** Your AI is the first helper you go to. Crop to the bit that matters.
2. **Going round in circles?** Start a fresh chat and explain where you’ve got to.
3. **Stuck for about ten minutes?** Ask a helper, or message Phil Bonhard on LBG Microsoft Teams. Say which step you’re on and what you can see.

> **Stop and ask first** if a screen unexpectedly asks for payment, a card or an upgrade, or if you’re unsure whether something is safe to use.

If connecting or publishing goes wrong

It asks for a password or token	Say no. Say “Don’t ask for a password or token. Use the one-time code instead.”
A browser tab opens anyway	Close it, and type the code at github.com/login/device where you’re already signed in.
You can’t sign in to GitHub	Use the method you signed up with (email, Google or Apple), or the device you signed up on.
A Mac pop-up about developer tools	Choose Install and wait. It can take a few minutes.
Windows can’t install Git or the GitHub tool	Ask it to explain the manual download, or ask for help.
The code ran out	Say “Please start the GitHub sign-in again”.
A 404 page	Wait a few more minutes and refresh. Still missing after 15? Screenshot it and ask your AI what’s wrong.

If your AI tool gets in the way

It asks to do something you don’t recognise	Say no, and ask for help.
It suggests extra features	Say “not now”, and tell it the one thing you want.
You’ve run out of usage	Stop, and don’t buy anything on the spot. Wait and try again later, and tell Phil if it stays stuck.
You can’t sign up for something	Skip it and use a different route. Don’t use a work account to get round a problem.
You still see the old version	Do a hard refresh. It can take up to 10 minutes.
You have no AI tool set up	Use the by-hand route, which needs no tool to publish.

Keep it safe

Please read this first

This is a personal learning exercise. Use fictional or public sample content only. Do not enter, upload or share any LBG data in AI tools, GitHub repositories, prompts, screenshots or published websites. More on staying safe.

Don’t use any of these

- Customer information
- Internal documents
- Passwords, keys and tokens
- Confidential business information
- Colleague information
- LBG or other internal code
- Screenshots of systems

> Stay with fictional or public sample content. The mortgage theme uses a clearly made-up bank, and no real banking connection or customer data is ever needed.

Five things worth knowing

1. Published websites may be visible to anyone who has, or guesses, the address.
2. Public GitHub repositories expose their files to everyone, including earlier versions.
3. AI tools handle data according to their own terms, your account and your settings. This event doesn't claim any tool is approved by LBG and can't guarantee confidentiality.
4. Never paste a password, API key or token into a prompt, a repository or the code of a web page.
5. Use your own accounts and make sure you keep access to your work after the day.

> **If you're not sure, ask before you share.** Message Phil Bonhard on LBG Microsoft Teams. If you realise you've shared something you shouldn't have, stop and tell Phil straight away, and you'll get help with the next step.

A little more detail

Personal accounts

A personal account keeps your work yours after the event, and keeps work identities out of public places. Usernames and published addresses are visible to others.

Screenshots

They can capture names, tabs, notifications and other windows. Share only a screenshot of your own sample product, and crop out everything else.

Private repositories

GitHub allows them on a free account, but free publishing with GitHub Pages needs a public one, and a page published from a private repository is still public.

Letting a tool do the work

Claude and Codex can create files, run commands and use your GitHub account. Read each permission request and say no to anything you don’t understand.

On the event site

The site asks for the event password. Dates and places there are for the current event.

Get ready
The checklist for before the day

Make it yours, then publish
The publishing steps

Plan your product
The full planning exercise

Phone or tablet route
The whole day on a small screen

Stay safe
The safety rules

Resources
Official guides and slides

Hello-world warm-up
Step-by-step for Claude and Codex

Project themes
The briefs and advanced routes

The day
Agenda, finish line and demos

LBG laptop route
Copilot and Figma Make

Get help
Who to ask and common fixes

You lead, the AI builds
Slides from the opening talk

Official guides

These open the provider's own site. Plans can change, so check before you sign up. We link to documentation, not products, and don't claim any tool is approved or endorsed.

GitHub: creating an account

GitHub Pages: what it is

Claude desktop app: get started

Codex: get started

Codex: plans and usage limits

Lovable: GitHub sync

Replit: Starter plan

GitHub Pages: create a site

GitHub CLI

Claude Code: plans that include it

Codex: approvals and security

Lovable: publishing

Lovable: credits and usage

> The planning prompt is adapted from Matt Pocock's grill-me skill. Installing it is optional.